Laboratory Work 3

Student: Ivaikina Darya

1. What is the fundamental difference between a container and a virtual machine?

The main difference is that a virtual machine runs its own operating system and kernel, while a container shares the host OS kernel and isolates the application and its dependencies. Containers are more lightweight, start faster, and require fewer resources than virtual machines. In this laboratory work, I used Docker to run my Flask REST API inside a container without creating a separate operating system.

2. Why does a Dockerfile first copy requirements.txt and install dependencies, and copy the code afterwards?

Docker builds images in layers and uses caching to avoid repeating unnecessary steps. The `requirements.txt` file is copied first because dependencies usually change less frequently than application code. If I modify `app.py` without changing the dependencies, Docker can reuse the cached installation layer. This reduces build time and makes the development process more efficient.

3. Why is a pod created through a Deployment rather than run “bare”? What happened when you deleted a pod?

A Deployment manages pods and ensures that the required number of replicas is always running. Unlike a standalone pod, a pod managed by a Deployment is automatically replaced if it is deleted. In this lab, I deployed three replicas of my Flask API. When I deleted one pod using `kubectl delete pod`, Kubernetes automatically created a new pod to restore the desired number of replicas. This demonstrated the self-healing capability of Kubernetes.

4. How does a Service find the right pods, and why must a service be stateless for it to be copied?

A Kubernetes Service uses labels and selectors to find the correct pods. In my project, both the Deployment and Service used the label `app: my-api`, allowing requests to be distributed among the matching pods. An application should be stateless so that each replica can process requests independently without relying on data stored in another pod. My `/time` endpoint is stateless, but the file-based `/jobs` queue from Lab 2 would require shared storage or an external queue to work reliably across multiple pods.